

Project



ID607001: Introductory Application Development Concepts

Assessment Information

Level	Credits	Assessment Type	Weighting
6	15	Individual	80%

Assessment Overview

In this individual assessment, you will design and develop a backend application using Express and a frontend application using SvelteKit.

Learning Outcome

At the successful completion of this course, learners will be able to:

1. Design and build secure applications with dynamic database functionality following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	22 May at 4.59 PM	1
Project	80%	26 June at 4.59 PM	1

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by 26 June at 4.59 PM.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of 50% across all assessments in ID607001: Introductory Application Development Concepts.

Submission

You must submit all application files via GitHub Classroom.

- Repository URL: https://classroom.github.com/a/8sCyquQ_
- Branch: Switch to the Project branch using the following command: `git checkout -b project`
- Git Ignore: If you do not have one, create a .gitignore using this resource - [Node.gitignore](#)
- Due Date: Sunday at 11.59 PM
- Late Penalty: 10% per day, rolling over at 12.00 AM

The latest application files in the Project branch will be used to mark against the marking rubric. Please test your applications before you submit. Partial marks may be given for incomplete functionality.

Authenticity

All parts of your submitted assessment must be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

AI Tools

Learning to use AI tools is an important skill. While AI tools are powerful, you must be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You must still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository README.md file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

This also applies to code snippets retrieved from StackOverflow and GitHub.

Failure to do this may result in a mark of zero for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with Otago Polytechnic policies. Learners can view policies on the Otago Polytechnic website located at (<https://www.op.ac.nz/about-us/governance-and-management/policies>)(<https://www.op.ac.nz/about-us/governance-and-management/policies>).

Extensions

Familiarise yourself with the assessment due date. Extensions will only be granted if you are unable to complete the assessment by the due date because of unforeseen circumstances outside your control. The length of the extension granted will depend on the circumstances and must be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in ID607001: Introductory Application Development Concepts.

Assessment Requirements - Backend Application using Express

The backend application marking rubric is available (here)([./marking-rubrics/backend-marking-rubric.md](#)).

Design Phase - Learning Outcome 1 (10 marks)

Before you start the Design Phase, read through the Development Phase requirements below. This will help you understand what you need to build and ensure your design document is comprehensive.

Once you understand the requirements, create a design document that outlines the structure of your application. This will help you plan your work effectively.

To move onto the Development Phase, the course lecturer must approve your design document.

Entity Relationship Diagram (ERD) (5 marks)

- Create an ERD using a digital tool of your choice that shows six models, their fields and relationships.
 - For each field, include its name, data type and constraints.
 - For enum fields, include the possible values.
-

API Endpoints (5 marks)

- In a table format, show the endpoints you will implement for your application.
- For each endpoint, include:
 - HTTP method
 - Endpoint URL
 - Brief description of what the endpoint does
 - Authentication requirement (if applicable)
 - Role-based access control requirement and roles
 - Path parameters (if applicable)
 - Query parameters (if applicable)
 - Body parameters (if applicable)

Here is an example of how to document an endpoint:

HTTP Method	Endpoint URL	Description	Authentication Required	Role-Based Access Control Required and Roles	Path Parameters	Query Parameters	Body Parameters
POST	/api/institutions	Create a new institution	Yes	Yes. Admin	None	None	name (string, required), region (string, required), country (string, required)

Development Phase - Learning Outcome 1 (25 marks)

Once the course lecturer has approved your design document, you can start developing your application. You should follow the requirements in your design document, but you can make changes as needed. If you make significant changes to your design document, please update it accordingly.

Project Management (2 marks)

In this phase, you will use the Agile software development methodology.

- You will decide the duration of your sprints and how to break down your work into sprints and tasks.
- Create a GitHub Project to manage your work. The project must include columns for Backlog, In Progress and Done.
- Create issues for each task and move them across the columns as you work on them.

Here is an example of how to break down your work into sprints and tasks:

Sprint	Tasks
X	Set up Express application, set up Prisma and PostgreSQL database, implement models, implement authentication endpoints, implement CRUD endpoints for one model

Database (1 mark)

- Use Prisma to interact with a PostgreSQL database in development, testing and production environments.

Models (3 marks)

- Implement six models with a minimum of four fields each excluding `id`, `createdAt`, and `updatedAt`.
- Include two enum fields across your models, each with at least two values.
- Implement four relationships between models:
 - At least one one-to-one relationship
 - At least two one-to-many relationship
 - One additional relationship of any type above

CRUD (10 marks)

- Implement the following endpoints: (4 marks)
 - CRUD operations (create, read all, read by ID, update and delete) for each model
 - Register, login and logout using token-based authentication
 - Health check that verifies the application's status, database connectivity and uptime
 - Catch-all endpoint
- Implement validation on create and update operations. (1 mark)
- Implement filtering, sorting and pagination on read all operations. (1 mark)
- Implement role-based access control with at least two roles. Each role must have distinct permissions. (1 mark)
- Implement content negotiation middleware to return responses in JSON format. (1 mark)
- Implement cache middleware for read all and read by ID operations for each model. (1 mark)
- Implement rate limiting middleware based on the user's role. For example, users with the role "X" may have a higher rate limit than users with the role "Y". (1 mark)

API Tests (4 marks)

Implement API tests for the following:

- CRUD operations for each model
- Register, login and logout
- Health check endpoint
- Catch-all endpoint
- Validation
- Filtering, sorting and pagination
- Permissions based on the user's role

API tests must be able to run against both development and production environments.

Scripts (2 marks)

Include scripts in the package.json file to:

- Run the application in development environment
 - Format your code
 - Lint your code
 - Create and run a PostgreSQL database in development environment
 - Create and run a PostgreSQL database for API tests
 - Create a database migration
 - Reset the database
 - Seed the database with five records for each model
 - Build the application for production
 - Run the API tests
-

Deployment (3 marks)

- Deploy the application to Render.
 - Verify that the deployed application is working correctly by testing the endpoints using Postman.
-

Code Quality and Best Practices - Learning Outcome 1 (15 marks)

When developing your application, you must follow best practices for code quality. This will help ensure that your code is maintainable, scalable and secure.

Code Organisation (3 marks)

- Use a clear and logical project structure that implements separation of concerns.
 - Implement separation of concerns by keeping the presentation and data access layers separate.
 - Write modular code by breaking down your application into smaller, reusable functions and modules.
-

Code Style and Formatting (2 marks)

- Code must be linted using ESLint.
 - Code must be formatted using Prettier.
 - Follow naming conventions.
 - Use meaningful variable, function and class names that clearly describe their purpose.
-

Error Handling (3 marks)

- Handle errors gracefully and provide meaningful error messages.
 - Use appropriate HTTP status codes.
 - Return consistent error response format across all endpoints.
 - Handle database errors and validation errors appropriately.
-

Security (3 marks)

- Store sensitive data in environment variables.
 - Include a `.gitignore` file to prevent committing unnecessary files and directories to GitHub.
 - Implement password hashing using a secure algorithm.
 - Use secure JWT practices.
-

Version Control (2 marks)

- Maintain a clean Git history with descriptive commit messages.
 - Use conventional commit messages.
 - Commit regularly with small, focused changes rather than large, monolithic commits.
-

Documentation (2 marks)

In `backend-documentation.md`, include the following:

- Project description and purpose
 - Installation and setup instructions
 - Environment variable configuration
 - How to run the application in development environment
 - How to run the API tests in testing environment
 - Deployment URL
-

Assessment Requirements – Frontend Application using SvelteKit

The frontend application marking rubric is available [here](#).

Design Phase – Learning Outcome 1 (5 marks)

Before you start the Design Phase, read through the Development Phase requirements below. This will help you understand what you need to build and ensure your design document is comprehensive.

Once you understand the requirements, create a design document that outlines the structure of your application. This will help you plan your work effectively.

To move onto the Development Phase, the course lecturer must approve your design document.

Wireframes (5 marks)

- Create wireframes using a digital tool of your choice that show the layout and structure of your application.
 - Include wireframes for at least six different pages:
 - Home page
 - Authentication pages
 - Dashboard page
 - List page for at least one model
 - Detail page for at least one model
 - Create/edit form page for at least one model
 - For each wireframe, include:
 - Page title and navigation elements
 - Key components and their placement
 - Form fields and buttons
 - Data display areas
 - Responsive layout considerations
-

Development Phase – Learning Outcome 1 (18 marks)

Once the course lecturer has approved your design document, you can start developing your application. You should follow the requirements in your design document, but you can make changes as needed. If you make significant changes to your design document, please update it accordingly.

Project Management (2 marks)

In this phase, you will continue using the Agile software development methodology.

- Use the same GitHub Project from the backend application.
- Create issues for each task and move them across the columns as you work on them.

Here is an example of how to break down your work into sprints and tasks:

Sprint	Tasks
X	Set up SvelteKit application, implement layout and navigation, implement authentication pages, create reusable form components

Pages and Routing (4 marks)

- Implement at least eight pages/routes:
 - Home page
 - Login page
 - Register page
 - Dashboard page
 - List page for at least two different models
 - Detail page for at least two different models
 - Create/edit form page for at least two different models
 - 404 error page

Components (4 marks)

- Implement at least eight reusable components:
 - Navigation component
 - Form input component(s)
 - Table or list component
 - Modal or dialog component
 - Loading indicator component
 - Alert component
 - Card component
 - At least one additional custom component relevant to your application

API Integration (4 marks)

- Integrate with your backend API to:
 - Handle user authentication (1 mark)
 - Fetch and display data for all six models (1 mark)
 - Create, update and delete records for all six models (1 mark)
 - Implement error handling for API requests (1 mark)

Form Handling and Validation (2 marks)

- Implement client-side form validation that matches backend validation rules.
- Display validation errors to users.
- Provide user feedback during form submission.

Deployment (2 marks)

- Deploy the application to a hosting platform. (1 mark)
- Verify that the deployed application is working correctly by testing core functionality. (1 mark)

Code Quality and Best Practices - Learning Outcome 1 (12 marks)

When developing your application, you must follow best practices for code quality. This will help ensure that your code is maintainable, scalable and follows Svelte/SvelteKit conventions.

Code Organisation (3 marks)

- Use a clear and logical project structure following Svelte/SvelteKit conventions.
 - Organise components, pages and utilities in appropriate directories.
 - Keep components focused and single-purpose.
-

Code Style and Formatting (2 marks)

- Code must be linted using ESLint.
 - Code must be formatted using Prettier.
 - Follow Svelte/SvelteKit naming conventions.
 - Use meaningful variable, function and component names that clearly describe their purpose.
-

Component Design (2 marks)

- Create reusable components that can be used across multiple pages.
 - Use props to make components flexible and configurable.
 - Avoid code duplication by extracting common functionality into shared components.
-

Error Handling and User Feedback (2 marks)

- Handle API errors gracefully and display meaningful error messages to users.
 - Provide clear feedback for user actions.
 - Implement proper error boundaries to prevent application crashes.
-

Version Control (2 marks)

- Maintain a clean Git history with descriptive commit messages.
 - Use conventional commit messages.
 - Commit regularly with small, focused changes rather than large, monolithic commits.
-

Documentation (1 mark)

In `frontend-documentation.md`, include the following:

- Project description and purpose
 - Installation and setup instructions
 - Environment variable configuration
 - How to run the application in development environment
 - Deployment URL
-

Author: Grayson Orr

Course: ID607001: Introductory Application Development Concepts